



INTERNATIONAL UNIVERSITY OF GRAND-BASSAM
Excellence • Accountability • Opportunity

SCHOOL OF SCIENCE AND ENGINEERING

CSC 4305 · Software Engineering · Fall 2026

Requirements IV — Writing Well, the Data Dictionary and the SRS

Six properties every requirement must pass, the words that betray a defect, the NOT line that settles arguments, and what to leave out of the specification on purpose

Session 10 of 28 · Friday 9 October 2026 · 10:00 – 11:20 AM

· Prof. Guillaume Sanje Mpacko, Ph.D.

Requirements IV — Writing Well, the Data Dictionary and the SRS

SESSION 10 OF 28

Friday, 9 October 2026 · 10:00 – 11:20 AM · Unit 2 · Requirements

LEARNING OBJECTIVES

- ✓ Apply the six properties — unambiguous, verifiable, necessary, feasible, atomic, traceable — to critique and repair a badly written requirement.
- ✓ Recognise the words that signal ambiguity, and recognise when the correct repair is deletion.
- ✓ Write a data-dictionary entry with meaning, identity, attributes, constraints and, most usefully, exclusions.
- ✓ Describe what a lightweight SRS contains, and decide consciously which sections an agile project should leave out.

FROM SESSION 9

Twenty-four requirements exist, each carrying its source, and eight of them have measurable fit criteria. Today is quality control on the sentences themselves — because a requirement that two competent readers understand differently has not yet done any work, however good its evidence.

Reconcile the backlog against the requirements

Applying Session 9. You now have two lists that describe the same system — a backlog and a requirement set. Until they are joined, neither can be trusted.

WHAT HAPPENS

1. **Tag every story.** Three pairs take a third of the backlog each. Against each story write the FR it realises — or mark it **orphan**, meaning it produces a requirement nobody has written down. **Six minutes.**
2. **Find the silent assumptions.** For each story in sprints 2 and 3, name any NFR or DR it quietly depends on. S-07 assumes NFR-01; what else? **Five minutes.**
3. **Two lists, out loud.** Orphan stories (build with no written requirement) and unrealised requirements (written, but no story). Both are defects. Decide what happens to each. **Four minutes.**

Output: the tagged backlog and the two gap lists, in the progression log, with a named owner against each gap.

Next session's slot: assembling the SRS — so bring the tagged backlog and your fit criteria.

1 · Six properties of a well-written requirement

U

Unambiguous

Two competent readers derive the same meaning.

✗ “handle errors appropriately”

V

Verifiable

A finite test or inspection decides pass or fail.

✗ “shall be reliable”

N

Necessary

Remove it and a stakeholder need goes unmet.

✗ added because it seemed a good idea

F

Feasible

Achievable within the known constraints.

✗ “shall never lose data”



Atomic

One requirement, one testable statement.

✗ “register and book and print”



Traceable

An identifier, and a recorded source.

✗ unnumbered prose

Words that signal ambiguity — treat each as a defect until proven otherwise

appropriate · adequate · efficient · user-friendly · robust · flexible · fast · easy · minimal · if possible · as required · etc. · and/or · **support** · handle · process

“**Support**” is the worst of them, because it sounds technical. “The system shall support offline use” commits to absolutely nothing: a system that shows an error message when offline has arguably supported it.

APPLIED TO SANTE-BASSAM

Six first drafts, diagnosed and repaired — every one written by a student team on this project

As drafted	Fails	Repaired as
“The system shall support offline use.”	Unambiguous • Verifiable	NFR-01: every function the clerk and CHW use in a clinic day (FR-01–09) works with no network; sync (FR-05) completes within ten minutes of connectivity returning.
“The system shall never lose data.”	Feasible	NFR-02 with its fit criterion: 0 partial writes out of 20 induced power failures. The residual risk — device destroyed before sync — is named in the SRS and accepted in writing by the nurse.
“Search should be fast and easy.”	Verifiable • Atomic	Two requirements. NFR-03 (95th percentile ≤ 2 s) and a separate usability criterion: the clerk finds nine of ten returning patients without help.
“The clerk shall register patients, book appointments and record outcomes.”	Atomic	FR-01, FR-07, FR-08 — three requirements, three tests. The <i>and</i> in a requirement is nearly always a seam.
“Patient names shall be validated appropriately.”	Unambiguous • Necessary	Deleted. Nobody asked for it, and many patients have a single name the clerk spells three ways. The real need was FR-02’s tolerance of variants.
“Records will be kept as required by law.”	Traceable • Verifiable	DR-03: retention for the statutory period; void, never delete; citing the article. Verified by inspecting the change log.

ROW FIVE IS THE ONE TO REMEMBER

The repair for an unnecessary requirement is deletion. Not rewording, not adding detail — removal. And the discipline to delete comes from one habit: ask *which stakeholder*, *which source*, and find none. Every requirement you carry is something a future student has to implement, test and maintain. A team that has never deleted a requirement has not yet been honest about one.

2 · The data dictionary

Half the arguments on a software project are two people using one word for two things. A data dictionary defines every domain term precisely — its meaning, its identity, its attributes, its constraints — and, most usefully of all, **what it is not**.

THE SIX FIELDS, AND WHY THE LAST ONE MATTERS MOST

DEFINITION

What the term means, in one or two sentences, in the language of the domain and not of the database.

IDENTITY

What makes two of them the same one. Which field is the identifier, whether it is reused, whether it is ever shown.

ATTRIBUTES

Each with required-or-optional, type and range.

CONSTRAINTS

What must always hold — and, just as important, what must *not* be assumed to hold.

SOURCE

The document, observation or regulation the definition came from.

NOT

The boundary. What this term excludes, and which neighbouring thing it is often confused with. **This is the line that settles arguments**, and it is the one students omit.

APPLIED TO SANTE-BASSAM

Three entries — Patient, Appointment, Visit

DATA-Dictionary.MD · PATIENT

TERM : Patient

DEFINITION : an individual registered at the post, whether or not they ever attended. Registration alone creates a Patient.

IDENTITY : patient_id, locally generated (HP-BSM-YYYY-XXXXXX); never reused; never shown to the patient. national_id assigned later, one-to-one (DR-01). card_number is searchable but NOT unique.

ATTRIBUTES : given_name, family_name required, 1-60 chars
birth_date required; year-only stored as 1 July
sex F, M, U
village one of the 23 (FR-11)
mother_name optional; primary lookup key in practice
phone optional
guardian required if under 18 (DR-02)

CONSTRAINT : (family_name, mother_name, village, birth_year) is NOT unique. Duplicates are DETECTED (FR-03), never prevented.

SOURCE : card analysis; obs. row 4; regulation art. 6

NOT : not a Visit and not an Appointment. A child brought by a neighbour IS a Patient; the neighbour is not.

APPOINTMENT

DEFINITION : a commitment that a Patient will attend on a date. Created by clerk or nurse; exists before any Visit.

IDENTITY : appointment_id; exactly one Patient.

ATTRIBUTES : scheduled_date (a DATE, not a time -- there are no slots)
reason (optional, 0-120)
status (booked, rescheduled, closed)

NOT : not a Visit. An Appointment that nobody attends still exists, and is why FR-08 has a "missed" outcome.

VISIT

DEFINITION : a patient's attendance at the post on a date, recorded by the nurse.

IDENTITY : visit_id; exactly one Patient; zero or one Appointment.

ATTRIBUTES : visit_date, outcome, follow_up_date (optional), recorded_by

CONSTRAINT : never deleted; voided with a reason (DR-03).

NOT : NOT a clinical record. No diagnosis, no treatment, no notes. This line defines the scope of the whole project.

The Visit NOT line is doing the most work in this document

“Not a clinical record” is what keeps diagnoses, prescriptions and clinical notes out of scope — and with them a whole category of regulation your team could not possibly satisfy this semester. Somebody will propose adding a notes field in week 10. That line is what you will quote back.

3 • The SRS — and what to leave out of it

A Software Requirements Specification is the stable home for the parts of the specification that must not move. On an agile project it is **distributed**, not abolished: the functional requirements live in the backlog, where they change every sprint, and everything that must *not* change every sprint lives in a short document.

§	Section (ISO/IEC/IEEE 29148)	Your decision	Why
1	Introduction, scope, definitions	Keep — short	Points to the data dictionary rather than repeating it
2	Overall description — environment, users, constraints, assumptions	Keep — this is the important one	Describe the post, the power, the devices, so that NFR-01, 02 and 08 are <i>obviously</i> necessary to any reader

3	Specific functional requirements	Omit — deliberately	The backlog carries these, with acceptance criteria. Duplicating them creates two versions that disagree by week 6
4	Non-functional requirements	Keep — with fit criteria	These outlive every sprint and decide the architecture
5	Domain requirements and external constraints	Keep — with the article cited	Non-negotiable, and nobody will restate them for you
6	Verification	Fold into §4	The fit criterion is the verification method
—	Appendices — data dictionary, traceability matrix	Keep	The dictionary settles arguments; the matrix is your test-writing backlog

WRITE DOWN WHAT YOU LEFT OUT, AND WHY

The omissions are the engineering judgement, so they are the part worth recording. Half a page at the front: why §3 lives in the backlog; why clinical notes, time-slot scheduling and multi-post scale are out of scope this semester. Without that page, a reader in February cannot tell a deliberate omission from an oversight — and neither will you.

SESSION 10 TAKEAWAY

Two competent readers must derive the same meaning, or it is not a requirement yet. Hunt the ambiguity words — “support” first — and remember that the repair for an unnecessary requirement is deletion. Define every domain term, and define what it is *not*, because that line is what stops the same argument recurring for fourteen weeks. Then keep the SRS short, keep the functional requirements in the backlog where they can move, and write down what you left out.

4 • In-class exercise • repair five requirements

Twelve minutes in three pairs, three minutes comparing. For each: **classify** it (FR / NFR / DR / not a requirement), **name the properties it fails**, and **rewrite it** specific and verifiable — with a fit criterion if it is an NFR. Five here; the other five are homework.

The five

1. The app shall be secure.
2. The system shall use PostgreSQL.
3. Sync shall happen quickly when the network comes back.
4. Patient records shall be deleted when requested.
5. The tablet app shall have a green submit button at the bottom right.

Statement **n**

Classify : FR / NFR / DR / not a requirement

Fails : which properties

Repaired : the rewritten requirement

Note : where implementation is over-specified, if it is

THE THREE THAT ARE MORE INTERESTING THAN THEY LOOK

#2 is a design decision wearing a requirement's clothes. The need behind it is portability, or a sponsor constraint — and it should say which. #4 contradicts DR-03: a stakeholder wish overruled by a domain requirement, which is the clearest example in the course of why domain requirements are a separate category. #5 over-specifies the implementation. The real need is *the CHW can submit one-handed while holding a child* — a usability requirement with a completely different fit criterion, and possibly a different button position.

5 · Homework · individual · due before Session 11 (Tuesday 13 October)

Download **CSC4305_HW10_Template.docx** from **CSC_4305_SWE-Assignments**,

TEMPLATE

fill it in, rename it **CSC4305_HW10_FirstName_LastName.docx** and commit it to **/homework/S10/** in your own private repository.

A **.docx** file. Not a PDF, not a link.

Q1 Repair the other five. Same four-line format as class.

(6) The clerk shall be able to do everything she does today. (7) The system must be easy for nurses with limited computer experience. (8) Reports shall be accurate. (9) The system shall handle up to 10,000 patients. (10) The follow-up list shall be available on Mondays.

Q2 Delete one. Go through the twenty-four requirements in Session 9 and nominate *one* for deletion. Name the stakeholder and source you could not find, and say what would actually be lost if it went.

If you conclude that all twenty-four are necessary, say so — but you must name the one you came closest to cutting, and why you kept it.

Q3 Write one data-dictionary entry for Village, User or SyncRecord — all six fields, to the standard of §2. The NOT line is marked separately, so do not skip it.

For User, the interesting question is whether a person and a role are the same thing. For SyncRecord, it is whether a failed upload is one.

Q4 Hunt the ambiguity words. Read your own team’s backlog and acceptance criteria. Find three places where one of the flagged words appears, quote each, and rewrite it.

Your own writing, not someone else’s. Everyone finds some.

Q5 What our SRS will not contain, and why. Half a page, your own opinion, using the table in §3 as a starting point. Name at least two things you would leave out that the table does not mention, and defend each in a sentence.

This is the paragraph that goes at the front of the team’s SRS. The best one from the six will be the one we use, with the author credited.

Q6 Reading. Sommerville, *Software Engineering* 10th ed., §4.4. Optional: ISO/IEC/IEEE 29148, clause 9.

Where this goes next. The SRS you are about to assemble is a baseline. From Session 11 on, a requirement that changes without a recorded reason is a defect. Tuesday adds the thing the SRS cannot express: **behaviour over time** — what happens at step 4 when the power fails, and what the clerk sees when two children match.

Key terms

unambiguous

verifiable

necessary

feasible

atomic

traceable

ambiguity words

data dictionary

the NOT line

SRS

ISO/IEC/IEEE 29148

baseline

Readings and references

- Sommerville, I. *Software Engineering*, 10th ed. — Chapter 4, §4.3–4.4.
- ISO/IEC/IEEE 29148:2018, *Systems and software engineering — Requirements engineering*.

- Robertson, S. & Robertson, J. *Mastering the Requirements Process*, 3rd ed.